

Main

- Artifacts
- Data Source
 - + Raw
 - + Ingested
 - Train
 - Test
 - + Transformed
 - X_train, X_test
 - Y_train, y_test (not yet encoded)

- Notebook
 - + EDA
 - + Preprocessed
- Extensions
 - + Loggings
 - + Exceptions
 - + Utils
 - Save object
 - Load object
 - Model Evaluation Classification (F1, Accuracy)
 - Model Evaluation Regression (R2, RMSE)
- Pipelines
 - + Ingestions
 - Define paths
 - Load df, train_test_split
 - Save train and test
 - + Transformation
 - Define path
 - Load train, test (ingested)
 - Define Pipeline and ColumnTransformer
 - Split x, y and transform x
 - Save X_train, X_test, y_train, y_test
 - + (Validation)
 - + Model training
 - Load X_train, x_test, y_train, y_test
 - Encode y
 - Define Models dictionary
 - Define Parameter dictionary
 - Tuning Hyperparameters
 - Evaluating model accuracy
 - Pick the model with largest accuracy and save model to pickle
 - + Full Process
 - Call Ingestion, transformation and training at once
 - + Auto prediction (for Flask framework)
 - Create prediction using given data (load pkl, transform and return predict) (**use predict_proba**)
 - Create Class to get data from input

Others

- .gitignore
- .env
- README
- Requirement.txt
- Dockerfile
- setup.py
- service.py (for bentoml service call api)
- Templates
 - + Home.html
 - + Prediction.html
 - + Result.html

Run

- app.py (stick with Flask framework), also called Full_process pipeline
- test.py (test for bug catching)

Applications

- Flask
 - + Set route /, /predict, /result
 - + Get input data from predict.html and post to model
 - + Run prediction and print in result.html
- Docker
 - + Create Dockerfile to implement building and running docker image
 - + Create docker image taking app and dependencies
- MongoDB (yet to implement)
- Dagshub
 - + Add dagshub path (owner and repo name) into modl_trainer
 - + Track and compare pushes metrics like on Mlflow
- BentoML
 - + Initialize model_trainer_demo as saving model to bentoml.sklearn
 - + Create Service and call API from call_bentoml_prediction to predict with input data
 - + Add bentoml.yaml
 - > bentoml build
 - + bentoml serve service.py:model_api --reload

- DVC (track and manage large data files)
- Github Action
- AWS
 - + Beanstalk
 - + ECR
 - + S3
- MIFlow
 - + Add into Model_trainer file
 - + Add metric class to contain F1, accuracy and precision
 - + Create mlflow_tracking to log result on the web
 - + Try with other models to compare metrics on the web